WEEKLY MINI PROJECT 03 · TEAM 4

Who Doesn't Know?

Work Package · Frontend A

Frontend | React + TS

PROJECT	TEAM LEADER	OWNER
Who Doesn't Know?	**Taima Watfeh**	**Yazan Shartouh**

ROLE	AREA
Frontend developer	**pages · components · api · mocks · Home · Lobby**

> *You build the frame every screen lives in, plus the first two screens players see.*

1 Your Work at a Glance

#	Task	Main file(s)
T1	Your API file: create / join / start	`api/lobby.api.ts`
T2	Your API file: state / me	`api/state.api.ts`
T3	Your mock data	`mocks/lobby.mock.ts`, `mocks/state.mock.ts`
T4	Session storage	`session/session.ts`
T5	Polling hook	`hooks/useGame.ts`
T6	Routing + status switch + error banner	`App.tsx`, `GameRoute.tsx`
T7	Home screen	`screens/HomeScreen.tsx`
T8	Lobby screen	`screens/LobbyScreen.tsx`

> **Use the previous frontend pattern:** pages for screens, components for reusable UI, api files for backend calls, mocks for independent development, and shared types for the contract.

2 Mock Data First

PHASE 1 · DAYS 1–3

Mocks ON

`VITE_USE_MOCKS=true`. Your api file returns data from **your own** mock file.

PHASE 2 · DAY 4

Mocks OFF

`VITE_USE_MOCKS=false`. The same api functions call the real backend.

PHASE 3 · DAY 4–5

Fix together

Run the flows on the real backend; fix any difference in your own files.

3 Your Tasks, Step by Step

T1 Your API file: create / join / start

`src/api/lobby.api.ts`

1. `createGame(displayName)` → `POST /games`.
2. `joinGame(roomCode, displayName)` → `POST /games/join`.
3. `startGame(session)` → `POST /games/:roomCode/start` with `{ playerId }`.
4. Each function: `if (USE_MOCKS) return mock.xxx(...)`, else `request(...)`.

```
// Pattern every frontend api file follows
import { request, USE_MOCKS } from './http';
import * as mock from '../mocks/game.mock';
import type { Session } from '../types';

export function askQuestion(s: Session, targetRef: number, text: string): Promise<{ ok: true }> {
  if (USE_MOCKS) return mock.askQuestion(targetRef, text);
  return request('POST', `/games/${s.roomCode}/questions`, { playerId: s.playerId, targetRef,
text });
}
```

✔ Done when: Home and Lobby work with mocks on and off without changing screen code.

T2 Your API file: state / me

`src/api/state.api.ts`

1. `getState(roomCode)` → `GET /games/:roomCode/state`.
2. `getMe(session)` → `GET /games/:roomCode/me?playerId=…`.

✔ Done when: the polling hook only calls these two functions.

T3 Your mock data

`src/mocks/lobby.mock.ts` · `src/mocks/state.mock.ts`

1. `lobby.mock.ts`: `createGame` returns `{ gameId: 1, roomCode: 'K7MD2', playerId: 'p_mock1', playerRef: 1 }`; `joinGame` rejects name "taken" with `mockError('Display name already used')` and code `XXXXX` with `'Game not found'`.
2. `startGame` rejects with "2 more players needed" if fewer than 4 mock players.
3. `state.mock.ts`: fixtures `lobby2Players`, `lobby5Players`, `questioningState`, `decisionState`, `votingState`, `finishedState`, and `meInformed`, `meHidden`, `meHost`.
4. `getState` returns the fixture chosen by `localStorage.mockState` (default `lobby2Players`), so anyone can preview any status.

✔ Done when: the whole app runs with `VITE_USE_MOCKS=true` and no backend.

T4 Session storage

`src/api/session.ts`

1. `saveSession(s)`, `loadSession(roomCode)`, `clearSession()` using `sessionStorage` (survives refresh, separate per tab: good for the demo).
2. Never render `playerId` on screen.

✔ Done when: refreshing the page keeps the player in the same game.

T5 Polling hook

`src/api/useGame.ts`

1. `useGame(roomCode)` loads the session, then calls `getState` + `getMe` every 2 s.
2. Returns `{ state, me, session, refresh, error, loading }`; `refresh()` fetches immediately.
3. Stop the interval on unmount.

✔ Done when: every screen updates within 2 s on all devices.

T6 Routing + status switch + error banner

`src/App.tsx` · `src/pages/GameRoute.tsx` · `src/components/ErrorBanner.tsx`

1. Routes: `/` → Home, `/room/:roomCode` → GameRoute, `/dev` → previews.
2. GameRoute: use `useGame` and render by `state.status`: lobby → Lobby, questioning → Game, round_decision → RoundDecision, voting → Voting, finished → Result. Pass `{ state, me, session, refresh }`.
3. No session or room not found → message + button back to Home. Never a blank page.

✔ Done when: changing `mockState` switches to the right screen.

T7 Home screen

`src/pages/HomePage.tsx`

1. Two cards: **Create game** (display name) and **Join game** (room code, auto uppercase, max 5 + display name).
2. On success: `saveSession` and navigate to `/room/:roomCode`. On error: show `error.message`.

✔ Done when: create and join work with mocks, errors visible.

T8 Lobby screen

`src/pages/LobbyPage.tsx`

1. Room code in **very large letters**; list of players with a host badge.
2. "{playersNeeded} more players needed" when `playersNeeded > 0`.
3. Start button only if `me.isHost`; calls `startGame(session)`, then `refresh()`; shows errors.

✔ Done when: host sees Start, others don't; starting with fewer than 4 shows the error.

4 Shared Files You Import (do not edit)

```
// frontend/src/api/http.ts  (Day-0 foundation, by Taima. Nobody edits it)
export const USE_MOCKS = import.meta.env.VITE_USE_MOCKS === 'true';
const BASE = import.meta.env.VITE_API_URL;

export async function request<T>(method: 'GET' | 'POST', path: string, body?: unknown): Promise<T> {
  const res = await fetch(BASE + path, { method, headers: { 'Content-Type': 'application/json' },
                                         body: body ? JSON.stringify(body) : undefined });
  const data = await res.json().catch(() => ({}));
  if (!res.ok) throw new Error(Array.isArray(data.message) ? data.message.join(', ') : data.message ??
'Request failed');
  return data as T;
}
export const mockDelay = <T,>(value: T, ms = 300) => new Promise<T>(r => setTimeout(() => r(value),
ms));
export const mockError = (message: string, ms = 300) =>
  new Promise<never>((_, rej) => setTimeout(() => rej(new Error(message)), ms));
```

```
// frontend/src/types.ts  (Day-0 foundation)
export interface Session { roomCode: string; playerId: string; playerRef: number; }
export interface ScreenProps {
  state: PublicGameState;      // from Yazan's polling hook
  me: PrivateRoleView;         // from Yazan's polling hook
  session: Session;            // send session to YOUR OWN api functions
  refresh: () => void;         // call after your action succeeds, to update immediately
}
```

5 Rules for You

- Only edit your own files listed above.
- Never decide turns, rounds, votes or the winner in the frontend: only show the backend state.
- Mocks must match `types.ts` field by field.

6 Presentation Part

Your part in the English presentation: **frontend architecture, polling, and how screens catch up after refresh**. Prepare 1–2 minutes and be ready to explain your own code.